



Санкт-Петербургский государственный университет
Кафедра системного программирования

Разработка фреймворка анализа повторов в документации в рамках проекта DocLine

Павел Михайлович Макаров, группа 24.M71-мм

Научный руководитель: к.ф.-м.н. Д.В. Луцив, доцент кафедры системного программирования

Рецензент: А. Г. Глазырин, инженер-программист 1 категории, АО «Эврика»

Санкт-Петербург
2026

- Документация является важной частью программного обеспечения. Но современные разработки часто сопровождаются дублированием текстов в документации, затрудняя её сопровождение
- Одно из решений проблемы — использование ПО для создания документации. Но насколько такие инструменты распространены?

Алгоритмы поиска повтора

- Сходство Жаккара (сходство двух последовательностей)
- Алгоритм шинглов (сравнение случайных подпоследовательностей)
- Сходство Левенштейна (кол-во операций преобразования)
- Алгоритм Рабина–Карпа (хэширование скользящих окон)

Существующие решения

- **Natural Language Toolkit** — набор библиотек и программ для анализа естественного языка
- **NiCad Clone Detector** — модульный инструмент анализа повторов
- **DocReuse** — простой детектор дублирования документации на Java
- **CloneDetective** — инструмент для исследований поиска клонов
- **Duplicate Finder** — инструмент анализа повторов

Вышеперечисленные разработки являются устаревшими

Целью работы является создание фреймворка для анализа повторов текста для документаций

Задачи:

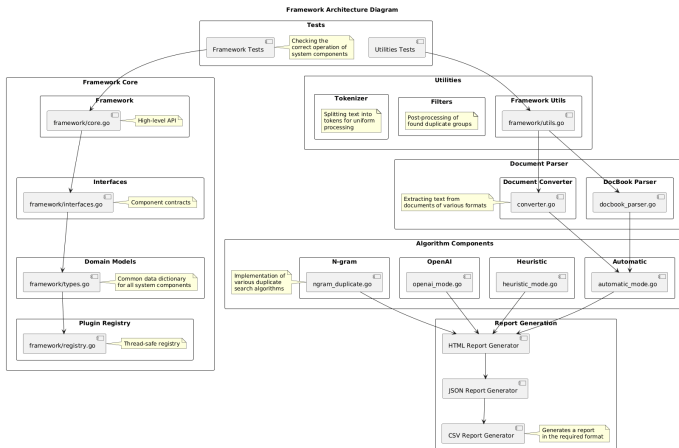
- Проанализировать существующие инструменты анализа текста
- Разработать архитектуру фреймворка
- Разработать фреймворк
- Реализовать тестирование и апробацию фреймворка

За основу была взята разработка Duplicate Finder

- Это инструмент с открытым исходным кодом для поиска похожих фрагментов текста в одном или нескольких файлах документации
- Является частью проекта DocLine
- Есть проблемы с внешними зависимостями, строгая платформенная зависимость

Архитектура решения

- Фреймворк реализован на языке программирования Go с модульной архитектурой с основой в виде ядра



Реализованные модули фреймворка

- Ядро фреймворка (интерфейсы, реестр модулей, конфиги)
- Алгоритмы поиска (автоматический, эвристический, по n-граммам, с помощью ИИ)
- Преобразование файлов, парсинг текста, создание отчётов

При необходимости, разработчик может создать свой модуль

Реализованные алгоритмы поиска

- Автоматический алгоритм
- Эвристический алгоритм
- Поиск по n-граммам
- Поиск с помощью ИИ

При необходимости, разработчик может создать свой алгоритм

Пример использования кода фреймворка

```
convertToDRL := true
archetypeLen := 3
strict := true

result, err := d.AnalyzeDocument("file.xml", "automatic", doctrine.AutomaticConfig{
    MinCloneLength: 5,
    MinGroupPower: 2,
    ConvertToDRL:   &convertToDRL,
    ArchetypeLength: &archetypeLen,
    StrictFilter:    &strict,
})
if err != nil {
    log.Fatal(err)
}
if err := d.GenerateReport(result, "html", "automatic.html"); err != nil {
    log.Fatal(err)
}
```

- Для тестирования реализован набор модульных и интеграционных тестов
- Для апробации документация 4 различных проектов проанализирована с помощью различных встроенных алгоритмов фреймворка

- Проведён анализ области, существующих решений и технологий
- Сформулированы требования к фреймворку
- Разработана архитектура фреймворка
- Выполнена реализация фреймворка
- Проведены тестирование и апробация фреймворка